Paso de Argumentos y Arreglos en C++

Curso Básico e Intermedio: Funciones, Valor, Referencia, Punteros y Arreglos

Alberto López García

Exposición y Taller Práctico · 4 Horas

La Gran Pregunta de la Sesión

El hilo conductor de las 4 horas

- **Una sola pregunta** sostiene las 4 horas de clase:
- ¿Qué recibe realmente una función en C++ cuando la llamas?
- Hay exactamente **tres respuestas posibles**:
- 1. Una **copia independiente** del valor original (paso por valor).
- 2. Un **alias directo** de la variable del llamador (paso por referencia).
- 3. Una **dirección de memoria** donde reside el dato (punteros y decaimiento de arreglos).
- Entender esto a nivel de memoria elimina el 90% de los errores y bugs en C y C++.

Estructura y Tiempos de la Exposición (4 Horas)

Distribución del tiempo minuto a minuto

Bloque	Tema / Actividad	Duración	Ejemplos src/
0	Apertura y Repaso de Funciones	10 min	01, 02, 03, 04
1	¿Cómo viaja un argumento? La Pila	15 min	05
2	Paso por Valor: Copias y Límites	25 min	06, 07, 08
3	Práctica Guiada 1: Funciones y Valor	15 min	ejercicios/practica-1
—	Receso Intermedio	10 min	—
4	Paso por Referencia: Alias y Eficiencia	30 min	09, 10, 11, 12
5	Punteros Básicos y Comparativa	20 min	13
6	Práctica Guiada 2: Referencias y Punteros	15 min	ejercicios/practica-2
—	Receso Intermedio	10 min	—
7	Arreglos Unidimensionales y Recorridos	25 min	14, 15, 16
8	Arreglos a Funciones: El Decaimiento	20 min	17
9	Arreglos Bidimensionales (Matrices)	20 min	18, 19
10	Repaso Final, Buenas Prácticas y Cierre	15 min	20, practica-3

Vocabulario Preciso: Parámetro vs. Argumento

Distinción fundamental para evitar ambigüedades

Término	¿Qué es?	¿Dónde se declara?	¿Dónde vive en memoria?
Parámetro	Variable formal declarada en la firma de la función	En la cabecera de la función	En el marco de pila de la función llamada
Argumento	Expresión o valor concreto transferido en la llamada	En el sitio de invocación (llamada)	En el marco del llamador (main u otra)

Regla de oro: El argumento inicializa al parámetro al entrar en la llamada.

Parámetro vs. Argumento en Código

```
// 'a' y 'b' son PARÁMETROS (variables locales de la función suma)
int suma(int a, int b) {
    return a + b;
}

int main() {
    int x = 10;
    int y = 20;

    // 'x' e 'y' son ARGUMENTOS (expresiones que producen valores)
    int res1 = suma(x, y);

    // Los argumentos pueden ser literales o expresiones complejas:
    int res2 = suma(5, 7);
    int res3 = suma(x * 2, y + 15);
}
```

El parámetro se crea y se destruye con la función. El argumento pertenece al llamador.

00

Anatomía de una Función en C++

30 minutos

- Definición, declaración y prototipos
- Tipo de retorno: void vs. valores calculados
- Ámbito (scope) y ciclo de vida de variables locales

Las Tres Partes de una Función

Estructura fundamental

- Una función es un bloque estructurado de instrucciones con nombre propio:
- 1. **Tipo de Retorno**: qué tipo de dato produce (`int`, `double`, `void`, etc.).
- 2. **Nombre o Identificador**: verbo o acción que describe lo que hace (`calcular_total`, `imprimir`).
- 3. **Lista de Parámetros**: insumos que requiere entre paréntesis `(tipo1 param1, tipo2 param2)`.
- **Cuerpo de la Función**: delimitado entre llaves `{ ... }` donde reside la lógica.
- **Instrucción `return`**: devuelve el control y el resultado calculado al llamador.

Demostración en vivo

01 · Función Básica: Definir vs. Llamar

```
./build/bin/bas_01_funcion_basica
```

- → Definir una función **no ejecuta nada**; es solo la receta escrita.
- → El código de la función cobra vida únicamente cuando es invocada.
- → La llamada puede usarse directamente dentro de expresiones matemáticas.

Prototipos vs. Definición

Problema: El compilador lee de arriba abajo

```
int main() {
    // ERROR de compilación:
    // 'cuadrado' was not declared
    int r = cuadrado(5);
}

int cuadrado(int n) {
    return n * n;
}
```

Solución: Prototipo previo

```
// PROTOTIPO (declaración anticipada)
int cuadrado(int n);

int main() {
    int r = cuadrado(5); // OK!
    return 0;
}

// DEFINICIÓN completa
int cuadrado(int n) {
    return n * n;
}
```

El prototipo avisa al compilador qué tipos entran y salen antes de que lea la definición.

Demostración en vivo

02 · Prototipo y Definición

```
./build/bin/bas_02_prototipo_y_definicion
```

→ Permite ordenar el archivo colocando `main()` arriba como índice de lectura.

→ Es la base técnica de los archivos de cabecera `.h` y `.hpp` en proyectos grandes.

Funciones void frente a Funciones con Retorno

¿Calcular un valor o ejecutar una acción?

- **Funciones con Retorno (`int`, `double`, etc.)**:
- Su misión principal es **calcular y transformar** información.
- Terminan obligatoriamente con `return expresion;`.
- Devuelven un único resultado al punto de invocación.
- **Funciones `void` (procedimientos)**:
- Su misión es **realizar una acción o efecto colateral** (imprimir, escribir en archivo).
- No devuelven ningún dato evaluable. El `return;` es opcional o actúa como salida anticipada.

Demostración en vivo

03 · void y Retorno de Valores

```
./build/bin/bas_03_void_y_retorno
```

→ Comprobación del uso de `void` para rutinas de impresión.

→ Salida anticipada con `return;` para validar datos incorrectos (cláusulas de guarda).

04 · Ámbito de las Variables (Scope)

```cpp
#include <iostream>

void f() {
    int x = 99; // 'x' local de f()
    std::cout << "f(): x = " << x << '\n';
}

int main() {
    int x = 10; // 'x' local de main()
    f();
    std::cout << "main(): x = " << x << '\n'; // sigue valiendo 10
}
```

Lo que nace dentro de las llaves { }, muere al llegar a la llave de cierre }.

01

¿Cómo Viaja un Argumento?

15 minutos

- La memoria del programa: Pila (Stack) y Montículo (Heap)
- Marcos de llamada (Stack Frames)
- Inspección de direcciones de memoria en vivo

La Pila de Llamadas (Call Stack)

El mecanismo físico de ejecución

- Cada vez que se llama a una función, la CPU reserva un bloque en la memoria llamado **Marco de Pila (Stack Frame)**.
- El marco de pila contiene:
- 1. Las variables locales de la función.
- 2. Los parámetros recibidos.
- 3. La dirección de retorno a la que debe volver la ejecución al terminar.
- La pila funciona en régimen **LIFO** (Last In, First Out):
- Al llamar, se **apila** un marco nuevo.
- Al retornar (`return`), el marco se **desapila** y esa memoria queda libre.

Demostración en vivo

05 · Llamadas Anidadas y Direcciones en la Pila

```
./build/bin/bas_05_llamadas_anidadas
```

→ Al anidar llamadas (`main -> nivel_1 -> nivel_2 -> nivel_3`), las direcciones **bajan** progresivamente.

→ Al retornar, los marcos se destruyen y las direcciones se reutilizan limpiamente.

→ Los parámetros son simplemente variables locales ubicadas dentro de ese marco.

02

Paso de Argumentos por Valor

25 minutos

- Mecánica: la copia exacta en el marco llamado
- Independencia absoluta de variables
- El problema clásico de swap_roto
- ¿Cuándo es el paso por valor la decisión correcta?

¿Qué es el Paso por Valor?

Copia independiente

- En el paso por valor, el argumento se evalúa y su resultado se **copia** en el parámetro.
- El parámetro es una variable **completamente nueva e independiente**.
- Propiedades clave:
- Tiene su **propia dirección de memoria** distinta a la del argumento.
- Cualquier modificación dentro de la función afecta **únicamente a la copia**.
- La variable original en el llamador queda **100% blindada e intacta**.

Demostración en vivo

06 · Demostración con Direcciones de Memoria

```
./build/bin/bas_06_paso_por_valor
```

- → Imprimimos `&original` en main y `&copia` dentro de la función: **son direcciones distintas**.
- → Asignar `copia = 999;` no altera el valor 42 original.
- → Para que un cálculo salga de una función por valor, debe usarse `return`.

07 · El Intercambio que NO Intercambia (swap_roto)

```
void intercambia_mal(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    // Dentro de la función, 'a' y 'b' SÍ cambiaron...
} // ...pero aquí mueren 'a' y 'b', que eran copias de la pila.

int main() {
    int x = 1, y = 2;
    intercambia_mal(x, y);
    std::cout << "x=" << x << " y=" << y << '\n'; // Imprime: x=1 y=2 (¡Sin cambios!)
}
```

La lógica del intercambio es perfecta. Lo que falló fue el canal de comunicación.

08 · ¿Cuándo Usar Paso por Valor?

El valor cuando conviene

- **Regla de oro de diseño**:
- El paso por valor **NO es un error ni una mala práctica**; es la opción óptima para:
- 1. **Tipos fundamentales pequeños**: `int`, `float`, `double`, `char`, `bool` (ocupan 1 a 8 bytes).
- 2. **Funciones matemáticas puras**: cálculo de áreas, raíces, conversiones de unidades.
- 3. **Copias de trabajo intencionales**: algoritmos donde modificamos el contador dentro de la función (ej. `cuenta_digitos`).
- 4. **Protección absoluta**: cuando el llamador exige que sus datos jamás sean alterados.

Práctica 1 · Funciones y Paso por Valor

15 minutos · ejercicios/practica-1-funciones-y-valor.md

1. Ejercicio 1: Predecir la salida de la función `misterio(a, b)` sin compilar.
2. Ejercicio 2: Encontrar por qué `duplica(n)` no modifica el original y corregirla con `return`.
3. Ejercicio 3: Implementar `perimetro_rectangulo`, `es_multiplo` e `imprime_cuenta_atras`.
4. Reto: Escribir `factorial(n)` iterativo usando el parámetro como copia de trabajo.

Receso Intermedio: 10 Minutos

Estirar las piernas, tomar agua y despejar la mente antes de entrar a referencias y punteros.

04

Paso de Argumentos por Referencia

30 minutos

- ¿Qué es una referencia? El concepto de alias
- Resolución definitiva de swap mediante referencias
- Devolver múltiples resultados mediante parámetros de salida
- La referencia constante (`const T&`): eficiencia y seguridad

09 · La Referencia: Un Segundo Nombre para un Mismo Objeto

Sintaxis y semántica

- En C++, `int& ref = original;` no reserva un nuevo entero:
- Crea un **alias** o apodo que referencia directamente a `original`.
- Reglas estrictas de las referencias:
- 1. Comparten **exactamente la misma dirección de memoria** (`&ref == &original`).
- 2. **Deben inicializarse obligatoriamente** al declararse.
- 3. **No se pueden reapuntar**: una vez atada a una variable, cualquier asignación modifica el valor del objeto referenciado.

Demostración en vivo

09 · Referencias y Direcciones en Vivo

```
./build/bin/bas_09_referencia_alias
```

- → Demostración: `&alias` y `&valor` imprimen exactamente el mismo valor hexadecimal.
- → Modificar `alias` cambia a `valor` instantáneamente.
- → Asignar `alias = otro;` no reasigna la referencia: copia el valor dentro de la variable ligada.

10 · El Swap Correcto con Referencias

```
// Recibe ALIAS de las variables del llamador
void intercambia(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
} // 'a' y 'b' modifican directamente a 'x' e 'y' de main

int main() {
    int x = 1, y = 2;
    intercambia(x, y);
    // ¡Ahora sí! Imprime: x=2, y=1
    std::cout << "x=" << x << " y=" << y << '\n';
}
```

Dos caracteres '&' en los parámetros cambiaron todo el comportamiento del programa.

11 · Devolver Varios Resultados (Parámetros de Salida)

Patrón de diseño clásico

- Una función en C++ tradicionalmente solo puede devolver **un único valor** con `return`.
- ¿Qué pasa si necesitamos calcular el cociente y el residuo a la vez?
- ¿O calcular el mínimo y el máximo de una sola pasada?
- Solución con referencias de salida:
- Los parámetros de entrada se leen.
- Los parámetros de salida van como referencias no constantes `int& out` para ser rellenados por la función.

Demostración en vivo

11 · Varios Resultados en una Sola Llamada

```
./build/bin/bas_11_varios_resultados
```

→ Función `division_entera(dividendo, divisor, cociente, resto)`.

→ Función `min_max(a, b, menor, mayor)`.

→ El llamador pasa sus variables y recibe los resultados calculados en un solo viaje.

12 · Referencia Constante (`const T&`): La Mejor Opción

Eficiencia + Seguridad

- ¿Qué ocurre cuando queremos la **eficiencia de no copiar**, pero la **seguridad de no modificar**?
- Usamos la **referencia constante**: `const Tipo& param`.
- Ventajas de `const T&`:
- 1. **Cero costo de copia**: no duplica memoria (vital para `std::string`, vectores o estructuras grandes).
- 2. **Seguridad garantizada**: el compilador prohíbe cualquier intento de modificación dentro de la función.
- 3. **Acepta literales y temporales**: permite llamadas como `imprime(42)` o `f(x + 5)`, cosa que `int&` rechaza.

Demostración en vivo

12 · const en Parámetros

```
./build/bin/bas_12_const_en_parametros
```

→ Demostración de cómo el compilador bloquea la asignación si se intenta modificar un parámetro `const`.

→ Comprobación de que `const T&` acepta tanto variables como expresiones temporales.

05

Punteros Básicos y Comparativa

20 minutos

- El puntero: una variable que almacena una dirección
- Operador de dirección & y operador de indirección *
- Comparativa práctica: Valor vs. Referencia vs. Puntero

13 · ¿Qué es un Puntero?

La tercera forma de comunicar funciones

- Un puntero es una variable cuyo valor es la **dirección de memoria** de otra variable.
- Dos operadores fundamentales:
- `&variable`: obtiene la dirección de memoria de la variable.
- `*puntero`: desreferencia o accede al dato que reside en esa dirección.
- Diferencias clave con las referencias:
- El puntero **puede ser nulo** (`nullptr`).
- El puntero **puede cambiar de dirección (reapuntarse)** a lo largo del tiempo.
- En la llamada, el llamador **hace explícito** que pasa una dirección: `f(&x)`.

Tabla Maestra: Valor vs. Referencia vs. Puntero

Matriz de decisión para el programador

Característica	Paso por Valor (`int n`)	Paso por Referencia (`int& n`)	Paso por Puntero (`int* p`)
¿Copia el dato?	**Sí** (duplica valor)	**No** (es un alias)	**No** (copia la dirección de 8B)
¿Puede modificar original?	**No**	**Sí** (o solo lectura con `const`)	**Sí** (desreferenciando `*p`)
Sintaxis en la llamada	`f(x)`	`f(x)` (transparente)	`f(&x)` (explícito con `&`)
¿Puede ser nulo?	No	No (siempre atado)	**Sí** (puede ser `nullptr`)
¿Cuándo se usa?	Tipos fundamentales pequeños	Uso general preferido en C++	Opcionalidad (`nullptr`) o arreglos

Demostración en vivo

13 · Punteros Básicos en Ejecución

```
./build/bin/bas_13_punteros_basicos
```

- → Declaración de puntero: `int* ptr = &x;`.
- → Lectura y modificación a través de `*ptr`.
- → Manejo seguro de punteros nulos con `if (ptr != nullptr)`.

Práctica 2 · Referencias y Punteros

15 minutos · ejercicios/practica-2-referencias-y-punteros.md

1. Ejercicio 1: Elegir la forma de paso adecuada (`T`, `T&`, `const T&`) para 5 firmas de funciones.
2. Ejercicio 2: Predecir la salida de un programa con referencias y asignaciones cruzadas.
3. Ejercicio 3: Implementar `divide(a, b, cociente, resto)` primero con referencias y luego con punteros.
4. Reto: Corregir `pon_a_cero(n)` para que modifique la variable del llamador.

Segundo Receso: 10 Minutos

Tomar café y estirarse. Al volver: el puente entre funciones y arreglos.

07

Arreglos Unidimensionales

25 minutos

- ¿Qué es un arreglo en memoria? Bloques contiguos
- Declaración, inicialización e indexación con corchetes
- Algoritmos de recorrido: suma, promedio, búsqueda y extremos
- Peligro mortal en C++: acceso fuera de rango (Out of Bounds)

14 · Anatomía de un Arreglo en C++

Contigüidad e indexación

- Un arreglo es una colección finita de elementos del **mismo tipo**, almacenados de forma **estrictamente contigua** en la memoria.
- Sintaxis: `Tipo nombre[TAMANO];` (ej. `int notas[5];`).
- La indexación comienza siempre en **cero** (`0` a `TAMANO - 1`).
- Cálculo directo de memoria:
- Dirección del elemento `i`: `dir(v[i]) = dir_base + i * sizeof(Tipo)`.
- Acceder a `v[i]` es una operación matemática inmediata de tiempo constante $\mathcal{O}(1)$.

Demostración en vivo

14 · Arreglo Básico y Direcciones Contiguas

```
./build/bin/bas_14_arreglo_basico
```

→ Inspeccionamos las direcciones de `v[0], v[1], v[2]`: cada una dista exactamente **4 bytes** de la anterior.

→ Inicialización con lista de valores `{10, 20, 30}` e inicialización en ceros `{0}`.

→ Recorrido clásico con bucle `for`.

15 · Algoritmos Fundamentales sobre Arreglos

Patrones de programación estándar

- Todo recorrido sobre un arreglo sigue el mismo esqueleto de bucle `for (int i = 0; i < n; ++i)`:
- 1. **Acumulación / Suma**: variable acumuladora inicializada en 0.
- 2. **Cálculo de la Media**: suma dividida entre el total de elementos `static_cast<double>(suma) / n`.
- 3. **Búsqueda de Extremos (Máximo y Mínimo)**: inicializar `maximo = v[0]` y comparar con cada elemento.
- 4. **Búsqueda Lineal**: recorrer hasta encontrar el valor y devolver su posición, o `-1` si no existe.

Demostración en vivo

15 · Recorridos y Estadísticas

```
./build/bin/bas_15_arreglo_recorridos
```

→ Cálculo de suma, promedio, valor máximo y conteo de aprobados.

→ Demostración de búsqueda de un elemento con parada temprana (`break`).

16 · El Peligro: Fuera de Rango (Undefined Behavior)

```cpp
int v[5] = {10, 20, 30, 40, 50}; // Índices válidos: 0, 1, 2, 3, 4

// ERROR GRAVE: el bucle llega hasta i = 5 (error de uno en uno)
for (int i = 0; i <= 5; ++i) {
    std::cout << v[i] << '\n';
}

// v[5] NO EXISTE: accede a memoria adyacente no reservada.
// En C++ esto NO lanza excepción automática; es COMPORTAMIENTO INDEFINIDO.
// Puede imprimir basura, corromper otras variables o abortar (Segfault).
```

En C++, el programador es el único responsable de no salirse de los límites del arreglo.

08

Arreglos como Parámetros: El Decaimiento

20 minutos

- La gran sorpresa: ¿un arreglo se copia al llamar a una función?
- El fenómeno del decaimiento a puntero (Array Decay)
- Por qué el tamaño SIEMPRE debe pasarse aparte
- Proteger arreglos de solo lectura con `const`

17 · La Gran Regla: Un Arreglo NUNCA se Copia

La bisagra teórica de la clase

- Cuando pasas un arreglo a una función en C y C++:
- **El arreglo NO se copia elemento por elemento.**
- El identificador del arreglo **decae automáticamente a un puntero** al primer elemento:
- `void f(int v[10])` es idéntico a `void f(int v[])` y a `void f(int* v)`.
- Consecuencias inmediatas:
- 1. La función recibe la **dirección de memoria de inicio**.
- 2. Cualquier modificación `v[i] = x;` **modifica el arreglo original del llamador**.
- 3. Dentro de la función, `sizeof(v)` mide 8 bytes (el puntero), **NO el tamaño del arreglo**.

Demostración en vivo

17 · Demostración del Decaimiento y sizeof

```
./build/bin/bas_17_arreglo_a_funcion
```

- → En main: `sizeof(datos)` vale **40 bytes** (10 enteros × 4 bytes).
- → Dentro de la función: `sizeof(v)` vale **8 bytes** (el tamaño de una dirección de 64 bits).
- → La función puede modificar `datos[0]` directamente porque tiene su dirección física.
- → Para protegerlo de modificaciones accidentales, se declara como `const int v[]`.

La Firma Estándar de una Función con Arreglos

```
// REGLA FUNDAMENTAL DE DISEÑO EN C/C++:
// Como el arreglo pierde su tamaño al decaer,
// el número de elementos DEBE pasarse como segundo parámetro.

// 1. Si la función solo lee el arreglo:
double promedio(const int v[], int n);

// 2. Si la función necesita modificar el arreglo:
void ordenar(int v[], int n);

// 3. En la llamada:
int notas[30];
double p = promedio(notas, 30); // se pasa el arreglo y su tamaño aparte
```

Fórmula universal: 'const T v[], int n' para leer; 'T v[], int n' para modificar.

09

Arreglos Multidimensionales (Matrices)

20 minutos

- Concepto de matriz: filas y columnas en memoria
- El orden Row-Major: almacenamiento lineal contiguo
- Fórmula de indexación física: i * COLUMNAS + j
- Pasar una matriz a una función: por qué el número de columnas es obligatorio

18 · Matrices en C++: Filas y Columnas

Representación conceptual vs. física

- Una matriz se declara como: `Tipo matriz[FILAS][COLUMNAS];`.
- Visualmente la pensamos como una tabla 2D con coordenadas `[fila][columna]`.
- Pero la memoria RAM de la computadora es **estrictamente lineal (1D)**.
- En C++, las matrices se almacenan en **Row-Major Order** (fila tras fila contigua):
- Primero se guardan todos los elementos de la Fila 0.
- Inmediatamente después, todos los elementos de la Fila 1, y así sucesivamente.

Demostración en vivo

18 · Matriz Básica y Fórmula de Indexación

```
./build/bin/bas_18_matriz_basica
```

- → Inspección de direcciones: `m[0][3]` y `m[1][0]` están pegados consecutivamente en memoria.
- → Fórmula que calcula el compilador entre bastidores:
- → $$\text{dir}(m[i][j]) = \text{dir_base} + (i \times \text{COLUMNAS} + j) \times \text{sizeof}(Tipo)$$
- → Recorrido estándar con dos bucles anidados `for (int i...) for (int j...)`.

19 · Pasar una Matriz a una Función: ¿Por qué [ ][C]?

```
// ¿Por qué esto COMPILA:
void imprime(const int m[][4], int filas);

// ...pero esto da ERROR de compilación:
// void imprime(const int m[][], int filas); // ¡ERROR!

// EXPLICACIÓN MATEMÁTICA:
// Para calcular dónde empieza la fila 'i', el compilador NECESITA saber
// cuántos elementos mide cada fila:
//     desplazamiento = (i * COLUMNAS + j)
// Sin el número de columnas, el compilador NO PUEDE saltar a la siguiente fila.
// El número de filas no hace falta para el salto; por eso se pasa como parámetro aparte.
```

Todas las dimensiones salvo la primera son obligatorias en la firma del parámetro.

Demostración en vivo

19 · Matriz como Parámetro en Vivo

```
./build/bin/bas_19_matriz_a_funcion
```

→ Implementación de `imprime_matriz(const int m[][4], int filas)`.

→ Cálculo de suma por filas y suma por columnas pasando matrices a funciones.

→ Protección con `const` para evitar alteraciones accidentales.

10

Repaso Final, Práctica 3 y Cierre

15 minutos

- Las 4 firmas esenciales del lenguaje en un solo programa
- Presentación de la Práctica 3 (tarea / laboratorio)
- Resumen para llevar y sesión de preguntas

Demostración en vivo

20 · El Gran Resumen en un Solo Código

```
./build/bin/bas_20_repaso_final
```

→ 1. `void f(int n)` -> COPIA. La función no altera el original.

→ 2. `void f(int& n)` -> ALIAS. La función modifica directamente al llamador.

→ 3. `void f(const int& n)` -> LECTURA. Cero copias, máxima protección.

→ 4. `void f(int v[], int n)` -> DIRECCIÓN. El arreglo decae y viaja sin copiarse.

Guía Rápida de Decisión para el Diseñador de Software

Árbol de decisiones

Situación	¿Qué tipo de paso elegir?	Ejemplo de firma
Dato primitivo pequeño (`int`, `double`, `bool`) que solo se lee	**Por valor**	`double seno(double x);`
Dato primitivo que la función debe modificar	**Por referencia**	`void duplica(int& x);`
Múltiples resultados calculados simultáneamente	**Por referencia (salida)**	`void divide(int a, int b, int& c, int& r);`
Tipo grande (`string`, clase, vector) solo para lectura	**Por referencia constante**	`void imprime(const std::string& msg);`
Colección fija de elementos (arreglo)	**Dirección + tamaño aparte**	`int suma(const int v[], int n);`
Puntero opcional que puede no existir	**Por puntero (revisar `nullptr`)**	`void configurar(const Config* cfg);`

Práctica 3 · Arreglos y Matrices (Para Llevar)

Laboratorio de consolidación

- Archivo de trabajo: `ejercicios/practica-3-arreglos.md`.
- Contenido de los ejercicios:
- 1. Estadísticas de calificaciones: media, máximo, mínimo y conteo de aprobados.
- 2. Modificar un arreglo desde una función (`suma_a_todos`).
- 3. Corregir el error de uno en bucles de recorrido.
- 4. Invertir un arreglo en su propio sitio (*in-place*) usando `intercambia` por referencia.
- 5. Suma de filas y columnas en una matriz bidimensional.
- 6. Reto opcional: cálculo de la matriz transpuesta M^T de 3×3.
- Soluciones de referencia disponibles en `ejercicios/soluciones/`.

¡Muchas Gracias!

Paso de Argumentos y Arreglos en C++ · Sesión Concluida

Alberto López García

Repositorio: exposicion-basica · Ejemplos compilados con CMake